

A top-down view of various school supplies arranged on a white surface. In the top left, a black mesh pen holder contains several pens and pencils. Below it, a pair of blue-handled scissors is open. To the right of the scissors is a red pencil, a white eraser, and a small blue sharpener. Further right is a black calculator with a small screen and various function buttons. Next to the calculator is a blue stapler. A blue ruler is positioned horizontally at the bottom of the frame. Above the ruler, there are two protractors (one blue, one white), a pink sticky note, and a silver pen. The background is a solid blue color with a vertical white line separating the supplies from the text area.

Week-9

# Introduction to Database Systems

Shah Nawaz



**Aggregate Functions  
Having, IN, NOT IN, Co-related  
Nested Query, EXISTS, NOT EXISTS,  
VIEW, ANY , ALL,  
Relational Algebra**



| ID   | Sales ID | Finance ID | Name   | Initials | City           | State | Email address   | Balance |
|------|----------|------------|--------|----------|----------------|-------|-----------------|---------|
| MD12 | 12345    | C5729      | Young  | N        | San Francisco  | CA    | young@xyz.com   | 100     |
| MD13 | 23324    | LA781      | Stills | S        | New Orleans    | LA    | stil@stil.org   | 200     |
| MD14 | 57657    | J7301      | Furay  | R        | Yellow Springs | OH    | buf@spring.com  | 150     |
| MD15 | 65461    | K8839      | Palmer | B        | New York       | NY    | bass@spring.com | 200     |
| ***  | ***      | ***        | ***    | ---      | ----           | ---   | -----           | -----   |

| ID   | Sales ID | Finance ID | Name   | Initials | City           | State | Email address   | Balance |
|------|----------|------------|--------|----------|----------------|-------|-----------------|---------|
| MD12 | 12345    | C5729      | Young  | N        | San Francisco  | CA    | young@xyz.com   | 100     |
| MD13 | 23324    | LA781      | Stills | S        | New Orleans    | LA    | stil@stil.org   | 200     |
| MD14 | 57657    | J7301      | Furay  | R        | Yellow Springs | OH    | buf@spring.com  | 150     |
| MD15 | 65461    | K8839      | Palmer | B        | New York       | NY    | bass@spring.com | 200     |
| ***  | ***      | ***        | ***    | ---      | ----           | ----  | -----           | -----   |



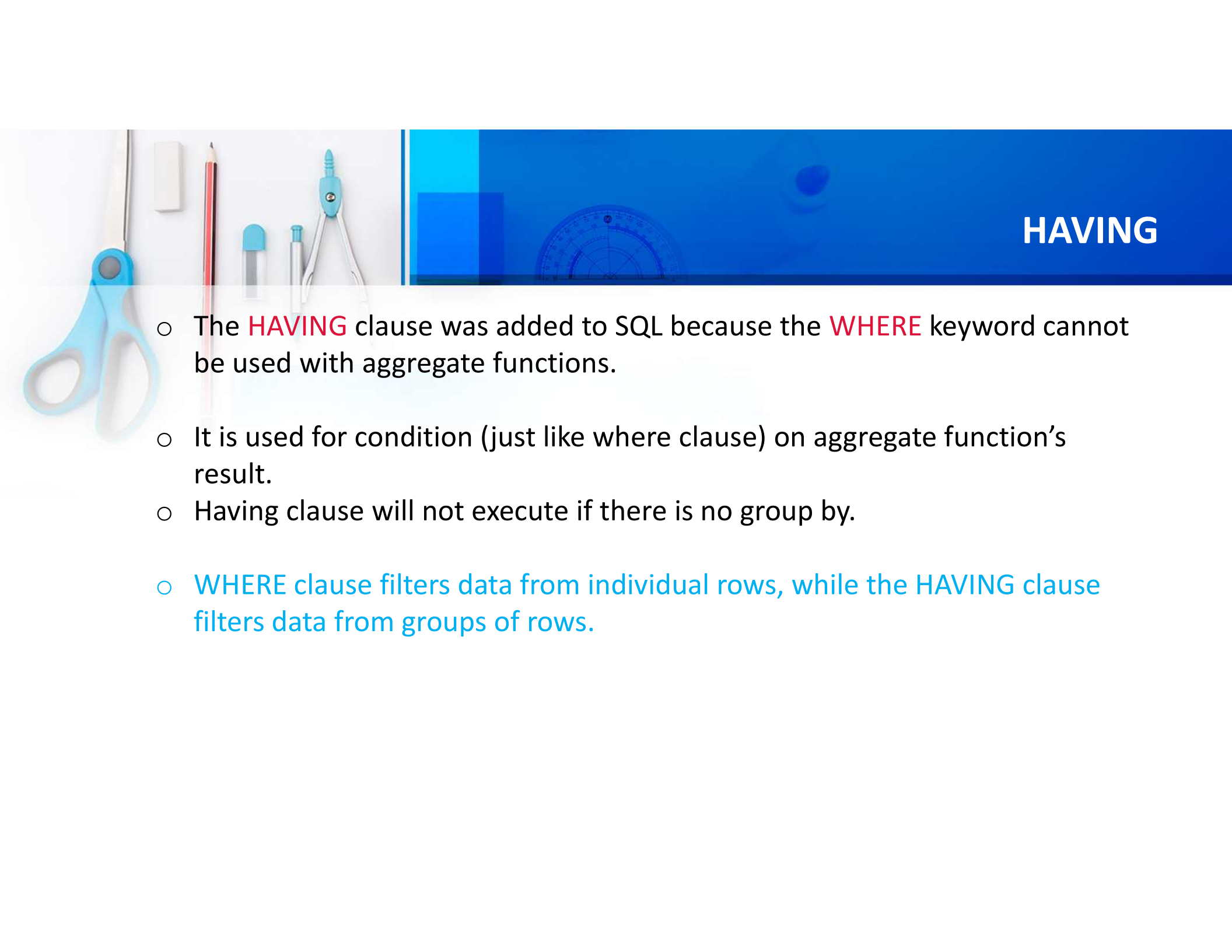
**No**, This SQL query has a syntax error. The alias TotalCount that is defined in the SELECT clause cannot be used in the WHERE clause directly.

[illegible]

| ID   | Sales ID | Finance ID | Name   | Initials | City           | State | Email address   | Balance |
|------|----------|------------|--------|----------|----------------|-------|-----------------|---------|
| MD12 | 12345    | C5729      | Young  | N        | San Francisco  | CA    | young@xyz.com   | 100     |
| MD13 | 23324    | LA781      | Stills | S        | New Orleans    | LA    | stil@stil.org   | 200     |
| MD14 | 57657    | J7301      | Furay  | R        | Yellow Springs | OH    | buf@spring.com  | 150     |
| MD15 | 65461    | K8839      | Palmer | B        | New York       | NY    | bass@spring.com | 200     |
| ***  | ***      | ***        | ***    | ---      | ----           | ---   | -----           | ---     |



**WHERE** keyword cannot be used with aggregate functions.



## HAVING

- The **HAVING** clause was added to SQL because the **WHERE** keyword cannot be used with aggregate functions.
- It is used for condition (just like where clause) on aggregate function's result.
- Having clause will not execute if there is no group by.
- **WHERE** clause filters data from individual rows, while the **HAVING** clause filters data from groups of rows.





**SELECT COUNT(ID) /  
FROM Customer  
GROUP BY State**



# Having Clause

| ID   | Sales ID | Finance ID | Name   | Initials | City           | State | Email address   | Balance |
|------|----------|------------|--------|----------|----------------|-------|-----------------|---------|
| MD12 | 12345    | C5729      | Young  | N        | San Francisco  | CA    | young@xyz.com   | 100     |
| MD13 | 23324    | LA781      | Stills | S        | New Orleans    | LA    | stil@stil.org   | 200     |
| MD14 | 57657    | J7301      | Furay  | R        | Yellow Springs | OH    | buf@spring.com  | 150     |
| MD15 | 65461    | K8839      | Palmer | B        | New York       | NY    | bass@spring.com | 200     |
| ---- | -----    | -----      | ---    | ---      | -----          | ----  | -----           | -----   |

| ID   | Sales ID | Finance ID | Name   | Initials | City           | State | Email address   | Balance |
|------|----------|------------|--------|----------|----------------|-------|-----------------|---------|
| MD12 | 12345    | C5729      | Young  | N        | San Francisco  | CA    | young@xyz.com   | 100     |
| MD13 | 23324    | LA781      | Stills | S        | New Orleans    | LA    | stil@stil.org   | 200     |
| MD14 | 57657    | J7301      | Furay  | R        | Yellow Springs | OH    | buf@spring.com  | 150     |
| MD15 | 65461    | K8839      | Palmer | B        | New York       | NY    | bass@spring.com | 200     |
| ---- | -----    | -----      | ----   | ---      | -----          | ----- | -----           | -----   |



```
SELECT *  
FROM Agents  
Where working_area = "London" OR  
working_area = "Mumbai" OR  
working_area = "Chennai";
```



The header features a light blue background on the left with various school supplies: a pair of blue-handled scissors, a white eraser, a red pencil, a blue sharpener, and a pair of compasses. On the right, a dark blue background contains a faint, stylized image of a globe and a protractor. The word 'IN' is written in large, white, sans-serif capital letters on the right side of the header.

# IN

```
SELECT *  
FROM agents  
WHERE working_area IN('London','Mumbai','Chennai');
```

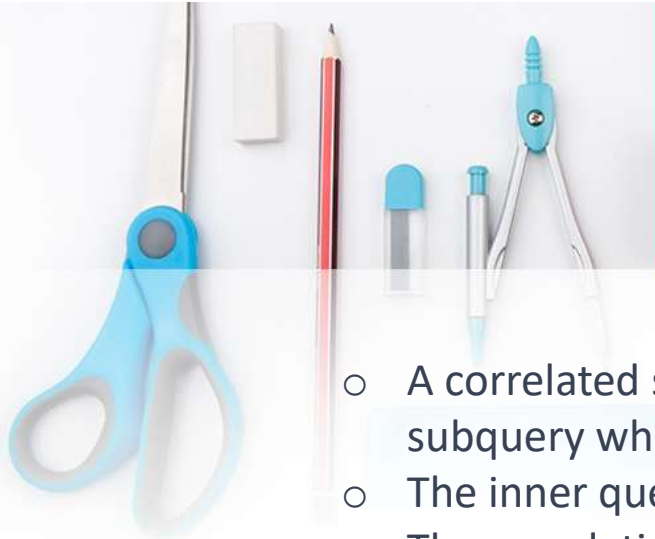
| agent_code | working_area | commission |
|------------|--------------|------------|
| A007       | Bangalore    | 0.15       |
| A005       | Brisban      | 0.14       |
| A001       | Bangalore    | 0.14       |
| A003       | London       | 0.12       |
| A008       | New York     | 0.12       |
| A002       | Mumbai       | 0.11       |
| A006       | London       | 0.15       |
| A004       | Torento      | 0.15       |
| A011       | Bangalore    | 0.15       |
| A010       | Chennai      | 0.14       |
| A009       | Hampshair    | 0.11       |
| A012       | San Jose     | 0.12       |



NOT IN

SELECT \*  
FROM agents  
WHERE commission NOT IN(.13, .14, .12);

| agent_code | working_area | commission |   |
|------------|--------------|------------|---|
| A007       | Bangalore    | 0.15       |   |
| A005       | Brisban      | 0.14       | X |
| A001       | Bangalore    | 0.14       | X |
| A003       | London       | 0.12       | X |
| A008       | New York     | 0.12       | X |
| A002       | Mumbai       | 0.11       |   |
| A006       | London       | 0.15       |   |
| A004       | Torento      | 0.15       |   |
| A011       | Bangalore    | 0.15       |   |
| A010       | Chennai      | 0.14       | X |
| A009       | Hampshair    | 0.11       |   |
| A012       | San Jose     | 0.12       | X |



## Co-related Nested Query

- A correlated subquery, also known as a correlated nested query, is a type of SQL subquery where the inner query references columns from the outer query.
- The inner query is executed once for each row processed by the outer query.
- The correlation between the inner and outer queries is established through references to columns in both queries.
- The basic structure of a correlated subquery looks like this:

```
SELECT column1, column2, ...  
FROM table1  
WHERE condition_operator  
(SELECT column3 FROM table2 WHERE condition_based_on_outer_query);
```



# EXISTS

The EXISTS condition in SQL is used to check whether the result of a correlated nested query is empty (contains no tuples) or not.

The result of EXISTS is a boolean value True or False. It can be used in a SELECT, UPDATE, INSERT or DELETE statement.

# EXISTS

Customers

| customer_id | lname   | fname   | website |
|-------------|---------|---------|---------|
| 401         | Singh   | Dolly   | abc.com |
| 402         | Chauhan | Anuj    | def.com |
| 403         | Kumar   | Niteesh | ghi.com |
| 404         | Gupta   | Shubham | jkl.com |
| 405         | Walecha | Divya   | abc.com |
| 406         | Jain    | Sandeep | jkl.com |
| 407         | Mehta   | Rajiv   | abc.com |
| 408         | Mehra   | Anand   | abc.com |

Orders

| order_id | c_id | order_date |
|----------|------|------------|
| 1        | 407  | 2017-03-03 |
| 2        | 405  | 2017-03-05 |
| 3        | 408  | 2017-01-18 |
| 4        | 404  | 2017-02-05 |

SELECT fname, lname  
FROM Customers WHERE EXISTS  
(SELECT \* FROM Orders WHERE Customers.customer\_id = Orders.c\_id);

| fname   | lname   |
|---------|---------|
| Shubham | Gupta   |
| Divya   | Walecha |
| Rajiv   | Mehta   |
| Anand   | Mehra   |

# NOT EXISTS

Customers

| customer_id | lname   | fname   | website |
|-------------|---------|---------|---------|
| 401         | Singh   | Dolly   | abc.com |
| 402         | Chauhan | Anuj    | def.com |
| 403         | Kumar   | Niteesh | ghi.com |
| 404         | Gupta   | Shubham | jkl.com |
| 405         | Walecha | Divya   | abc.com |
| 406         | Jain    | Sandeep | jkl.com |
| 407         | Mehta   | Rajiv   | abc.com |
| 408         | Mehra   | Anand   | abc.com |

Orders

| order_id | c_id | order_date |
|----------|------|------------|
| 1        | 407  | 2017-03-03 |
| 2        | 405  | 2017-03-05 |
| 3        | 408  | 2017-01-18 |
| 4        | 404  | 2017-02-05 |

SELECT fname, lname  
FROM Customers WHERE NOT EXISTS  
(SELECT \* FROM Orders WHERE Customers.customer\_id = Orders.c\_id);

| lname   | fname   |
|---------|---------|
| Singh   | Dolly   |
| Chauhan | Anuj    |
| Kumar   | Niteesh |
| Jain    | Sandeep |



In SQL, a view is a virtual table based on the result of a SELECT query. It does not store the data itself but provides a way to represent the result of a query as if it were a table.

Views are useful for [simplifying complex queries](#), providing a [layer of security](#) by [restricting access to certain columns](#) or rows, and abstracting the underlying structure of the database.

Views provide a way to encapsulate complex queries, control access to data, and present a simplified interface to users or applications interacting with the database.



## View

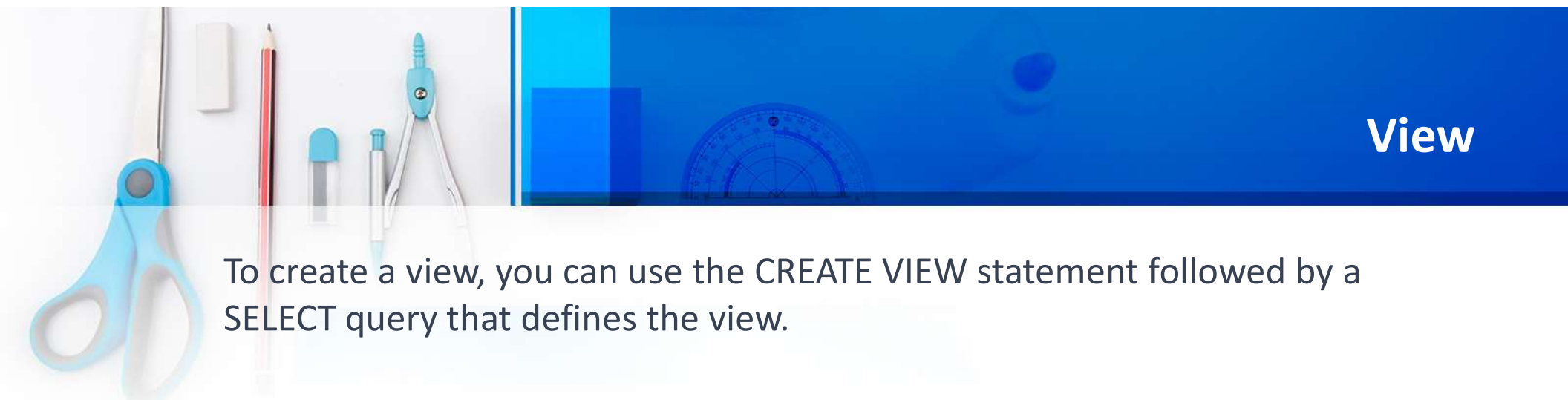
To create a view, you can use the CREATE VIEW statement followed by a SELECT query that defines the view.

Creating View:

```
CREATE VIEW my_view AS  
SELECT column1, column2  
FROM my_table  
WHERE condition;
```

Querying from View:

```
SELECT * FROM my_view;
```



## View

To create a view, you can use the CREATE VIEW statement followed by a SELECT query that defines the view.

Creating View:

```
CREATE VIEW employee_view AS  
SELECT employee_id, first_name, last_name  
FROM employees  
WHERE department_id = 10;
```

Querying from View:

```
SELECT * FROM employee_view;
```

The header features a light blue background on the left with various stationery items: a pair of blue-handled scissors, a white eraser, a red pencil, a blue sharpener, and a pair of compasses. On the right, a dark blue background contains faint, semi-transparent geometric shapes, including a square, a circle, and a protractor.

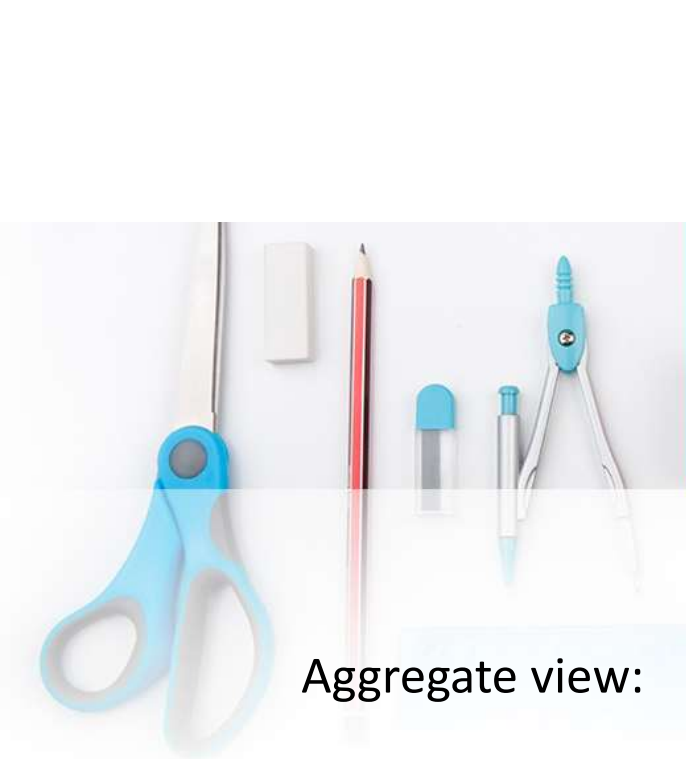
## View

Deleting View:

```
DROP VIEW employee_view
```

Joining tables:

```
CREATE VIEW order_details AS  
SELECT orders.order_id, customers.customer_name, order_date  
FROM orders  
JOIN customers ON orders.customer_id = customers.customer_id;
```



Aggregate view:

```
CREATE VIEW product_summary AS  
SELECT category, AVG(price) AS avg_price, COUNT(*) AS total_products  
FROM products  
GROUP BY category;
```



## ANY

**ANY** keyword is used in conjunction with a comparison operator to **compare a value to any value in a set of values**.

It is often **used in subqueries** and conditions to check if a condition is true for at least one element in a set.

```
SELECT * FROM Teachers  
WHERE age < ANY ( SELECT age FROM Students );
```

Here, the SQL command selects rows if age in the outer query is less than any age in a subquery.



# ANY

Table: Teachers

| id | name  | age |
|----|-------|-----|
| 1  | Peter | 32  |
| 2  | Megan | 43  |
| 3  | Rose  | 29  |
| 4  | Linda | 30  |
| 5  | Mary  | 41  |

Table: Students

| id | name  | age |
|----|-------|-----|
| 1  | Harry | 23  |
| 2  | Jack  | 42  |
| 3  | Joe   | 32  |
| 4  | Dent  | 23  |
| 5  | Bruce | 40  |

```
SELECT *  
FROM Teachers  
WHERE age < ANY (  
  SELECT age  
  FROM Students  
);
```

| id | name  | age |
|----|-------|-----|
| 1  | Peter | 32  |
| 3  | Rose  | 29  |
| 4  | Linda | 30  |
| 5  | Mary  | 41  |



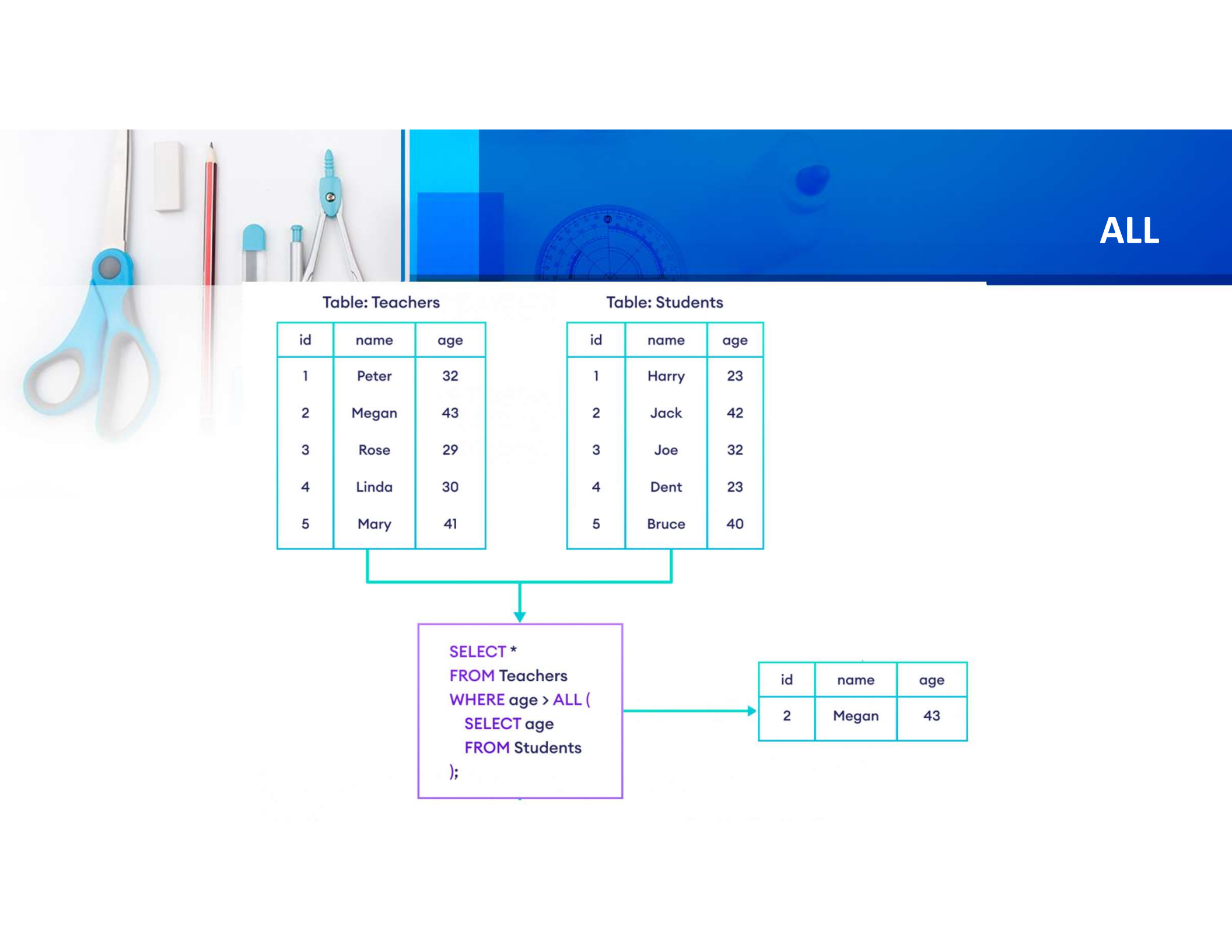
## ALL

SQL ALL compares a value of the first table with all values of the second table and returns the row if there is a match with all values.

For example, if we want to find teachers whose age is greater than all students, we can use

```
SELECT * FROM Teachers  
WHERE age > ALL ( SELECT age FROM Students);
```

Here, if the teacher's age is greater than all student's ages, the corresponding row of the Teachers table is selected.



# ALL

Table: Teachers

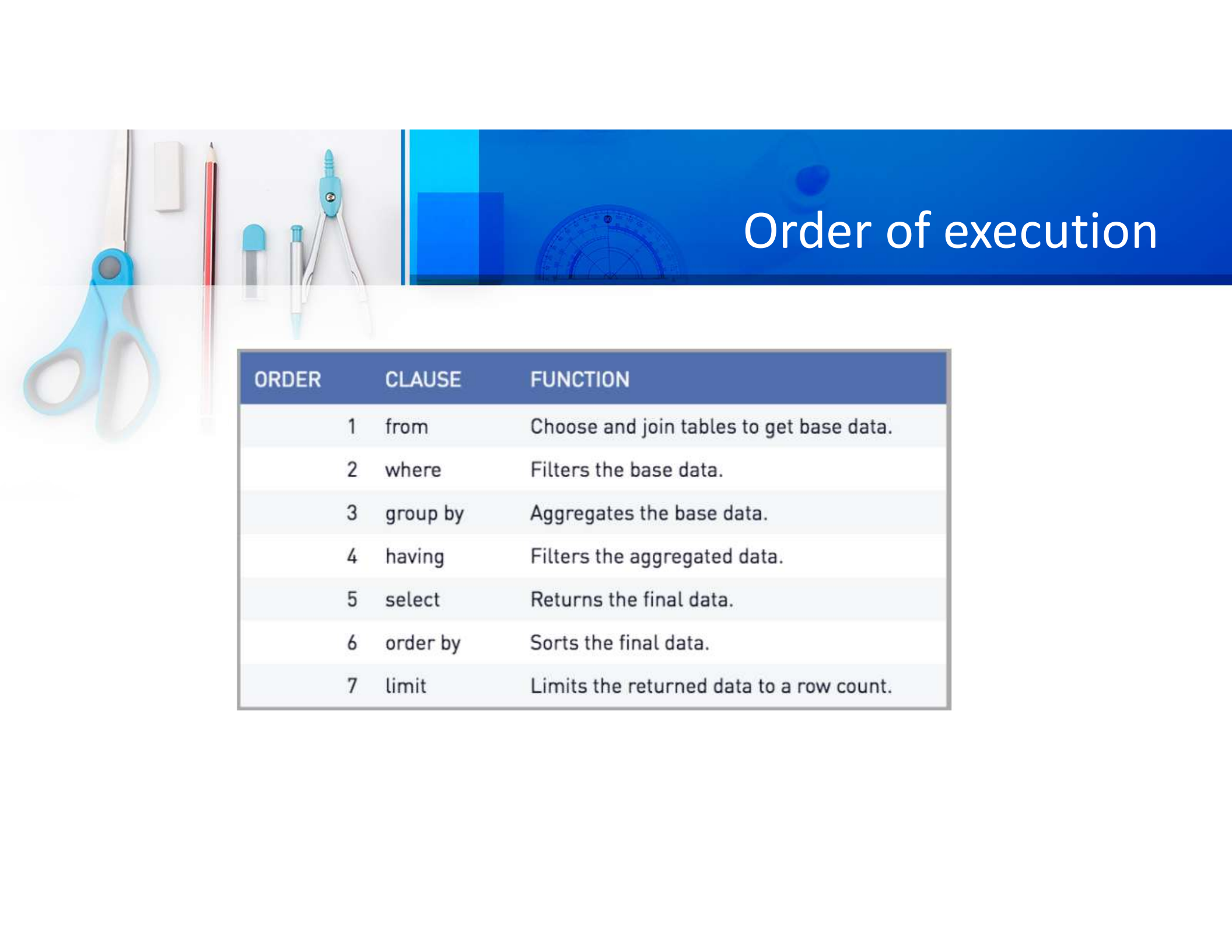
| id | name  | age |
|----|-------|-----|
| 1  | Peter | 32  |
| 2  | Megan | 43  |
| 3  | Rose  | 29  |
| 4  | Linda | 30  |
| 5  | Mary  | 41  |

Table: Students

| id | name  | age |
|----|-------|-----|
| 1  | Harry | 23  |
| 2  | Jack  | 42  |
| 3  | Joe   | 32  |
| 4  | Dent  | 23  |
| 5  | Bruce | 40  |

**SELECT \***  
**FROM Teachers**  
**WHERE age > ALL (**  
    **SELECT age**  
    **FROM Students**  
**);**

| id | name  | age |
|----|-------|-----|
| 2  | Megan | 43  |



# Order of execution

| ORDER | CLAUSE   | FUNCTION                                 |
|-------|----------|------------------------------------------|
| 1     | from     | Choose and join tables to get base data. |
| 2     | where    | Filters the base data.                   |
| 3     | group by | Aggregates the base data.                |
| 4     | having   | Filters the aggregated data.             |
| 5     | select   | Returns the final data.                  |
| 6     | order by | Sorts the final data.                    |
| 7     | limit    | Limits the returned data to a row count. |

The header features a white background on the left with various school supplies: a pair of blue-handled scissors, a white eraser, a red pencil, a blue sharpener, a blue pen, and a pair of compasses. To the right, a blue gradient background contains a faint, light blue geometric diagram of a circle with internal lines and a semi-circular protractor-like shape. The text 'Practice Queries' is written in white on the right side of the blue background.

# Practice Queries

1. Show average salary of all employees.
2. Show total number of departments.
3. Show sum of hours spent on all projects.
4. Show average salary of each department if average salary is greater than 40,000.
5. Show minimum salary of each department if minimum salary is less than 20,000.
6. Show total number of employees of each department with department name.
7. Show number of dependent of each employee.
8. Show count of projects of all departments in ascending order by department name.
9. Show count of total locations of each department which are located in Lahore.
10. For each project show project name and total number of employees working on it.

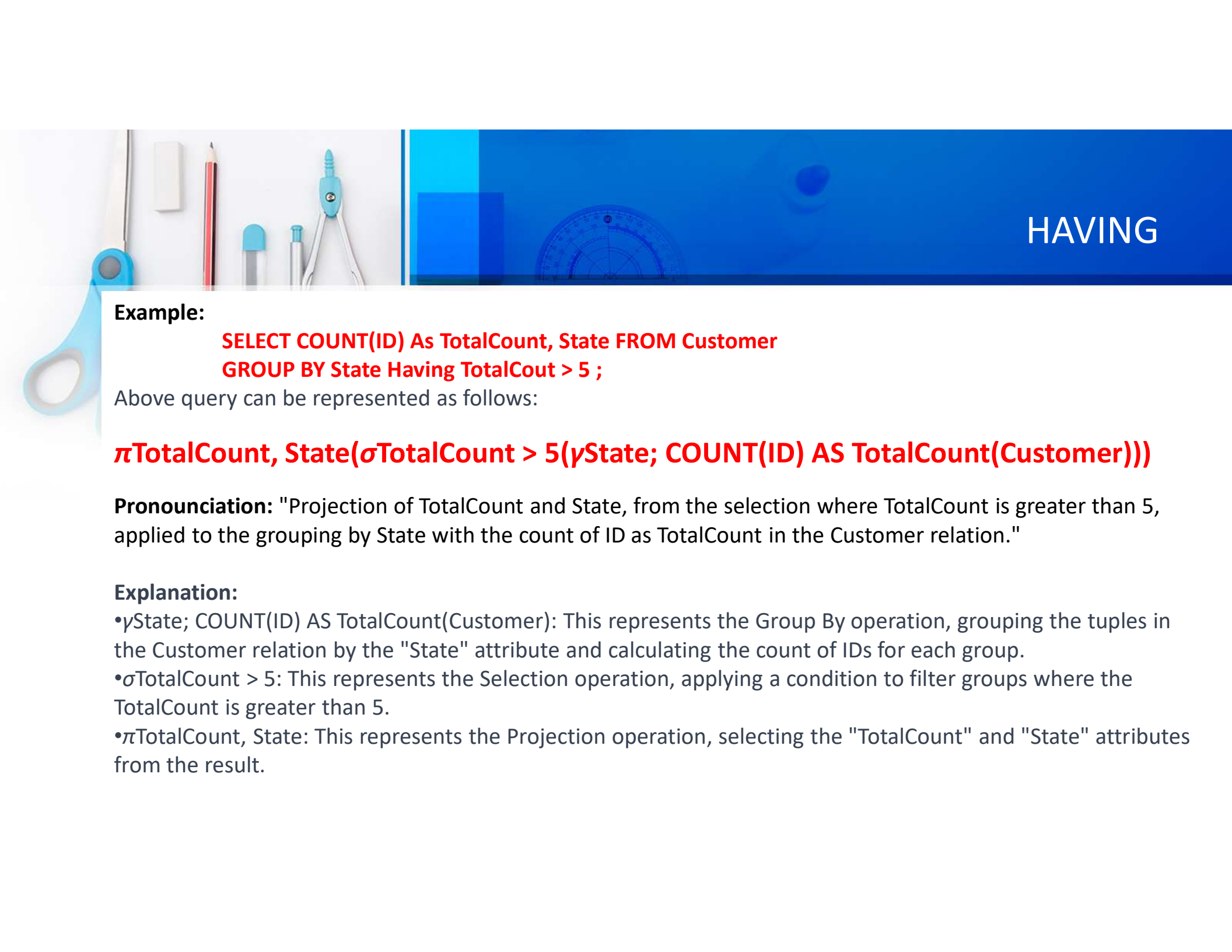


# Relational Algebra



## HAVING

In relational algebra, there isn't a specific symbol dedicated to the HAVING clause, as relational algebra primarily focuses on basic operations like selection ( $\sigma$ ), projection ( $\pi$ ), join ( $\bowtie$ ), and grouping ( $\gamma$ ). The HAVING clause is a part of SQL and is used in combination with the GROUP BY clause to filter the results of a grouped query based on a specified condition.



## HAVING

### Example:

```
SELECT COUNT(ID) AS TotalCount, State FROM Customer  
GROUP BY State Having TotalCout > 5 ;
```

Above query can be represented as follows:

**$\pi_{\text{TotalCount, State}}(\sigma_{\text{TotalCount} > 5}(\gamma_{\text{State; COUNT(ID) AS TotalCount}}(\text{Customer})))$**

**Pronunciation:** "Projection of TotalCount and State, from the selection where TotalCount is greater than 5, applied to the grouping by State with the count of ID as TotalCount in the Customer relation."

### Explanation:

- $\gamma_{\text{State; COUNT(ID) AS TotalCount}}(\text{Customer})$ : This represents the Group By operation, grouping the tuples in the Customer relation by the "State" attribute and calculating the count of IDs for each group.
- $\sigma_{\text{TotalCount} > 5}$ : This represents the Selection operation, applying a condition to filter groups where the TotalCount is greater than 5.
- $\pi_{\text{TotalCount, State}}$ : This represents the Projection operation, selecting the "TotalCount" and "State" attributes from the result.



The header features a light blue background on the left with various school supplies like a pencil, eraser, and compass. On the right, there's a dark blue background with a faint circular pattern. The text 'IN' is displayed in white on the dark blue background.

# IN

In relational algebra, the IN operator from SQL is typically expressed using a combination of operations. Let's consider a scenario where you want to retrieve all orders from the "Orders" relation where the customer ID is in a specified set {101, 102, 105}. The SQL query for this is:

**SELECT \* FROM Orders WHERE CustomerID IN (101, 102, 105);**

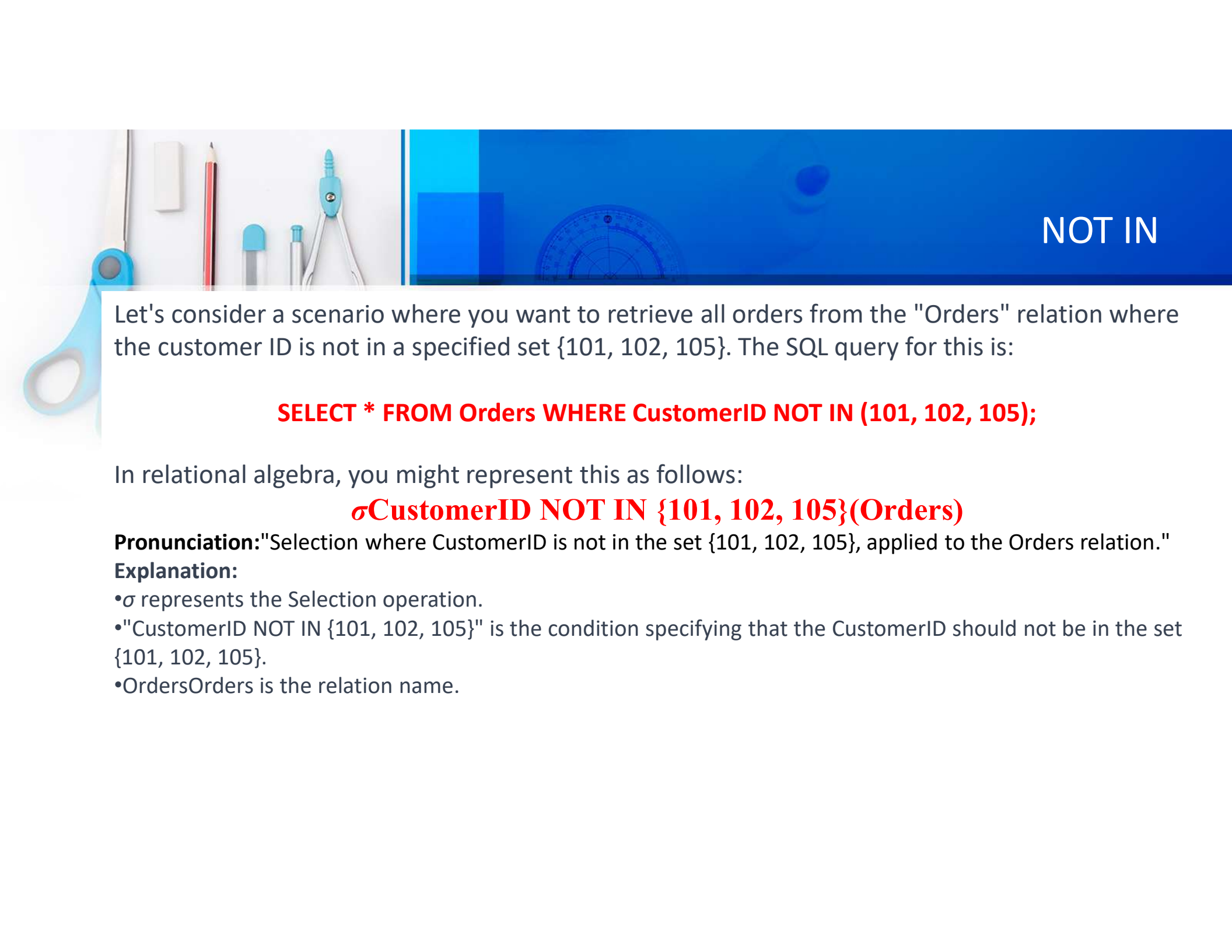
In relational algebra, you might represent this as follows:

**$\sigma_{\text{CustomerID} \in \{101, 102, 105\}}(\text{Orders})$**

**Pronunciation:** "Selection where CustomerID is in the set {101, 102, 105}, applied to the Orders relation."

**Explanation:**

- $\sigma$  represents the Selection operation.
- " $\text{CustomerID} \in \{101, 102, 105\}$ " is the condition specifying that the CustomerID should be in the set {101, 102, 105}.
- Orders is the relation name.



## NOT IN

Let's consider a scenario where you want to retrieve all orders from the "Orders" relation where the customer ID is not in a specified set {101, 102, 105}. The SQL query for this is:

```
SELECT * FROM Orders WHERE CustomerID NOT IN (101, 102, 105);
```

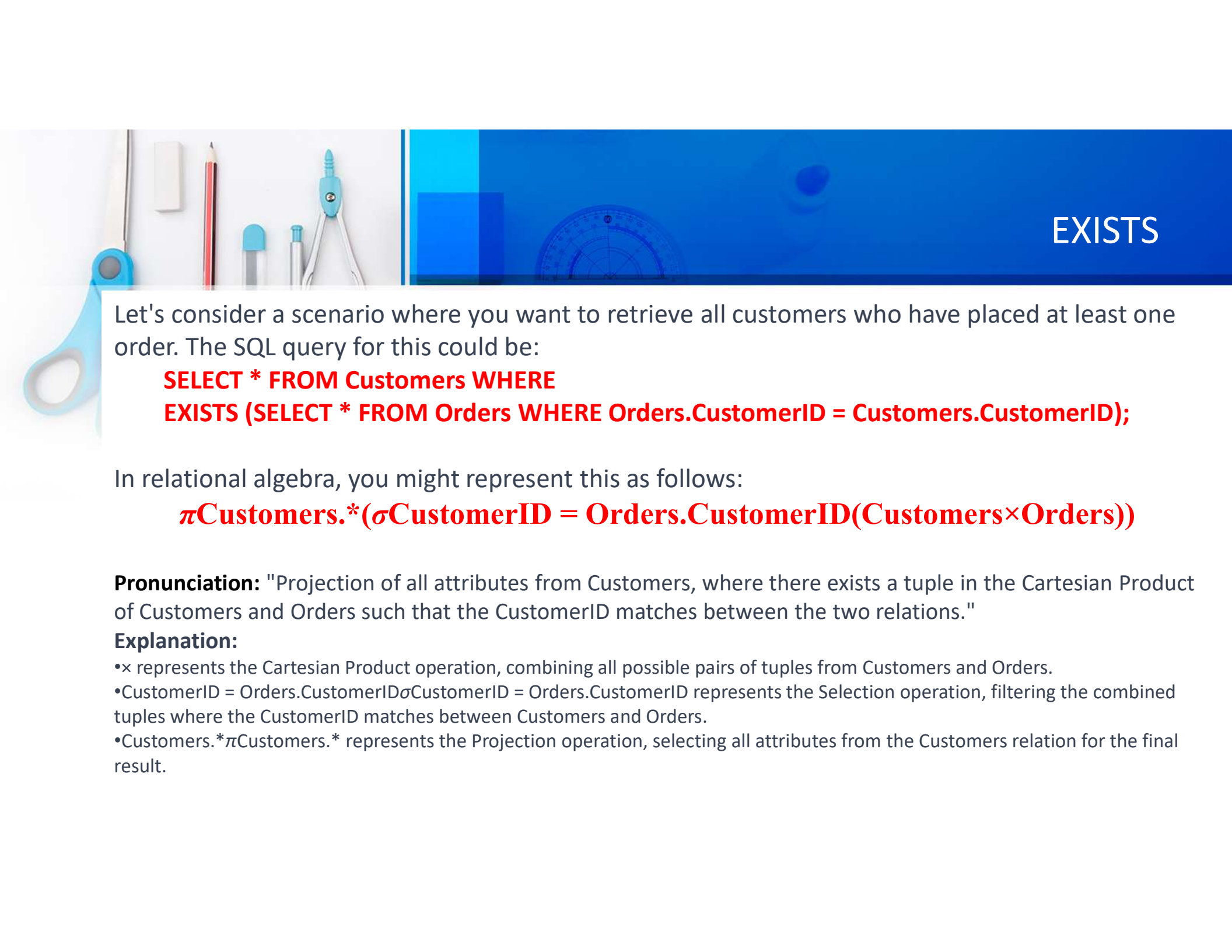
In relational algebra, you might represent this as follows:

```
 $\sigma_{\text{CustomerID NOT IN \{101, 102, 105\}}(\text{Orders})$ 
```

**Pronunciation:** "Selection where CustomerID is not in the set {101, 102, 105}, applied to the Orders relation."

**Explanation:**

- $\sigma$  represents the Selection operation.
- "CustomerID NOT IN {101, 102, 105}" is the condition specifying that the CustomerID should not be in the set {101, 102, 105}.
- Orders is the relation name.



## EXISTS

Let's consider a scenario where you want to retrieve all customers who have placed at least one order. The SQL query for this could be:

```
SELECT * FROM Customers WHERE  
EXISTS (SELECT * FROM Orders WHERE Orders.CustomerID = Customers.CustomerID);
```

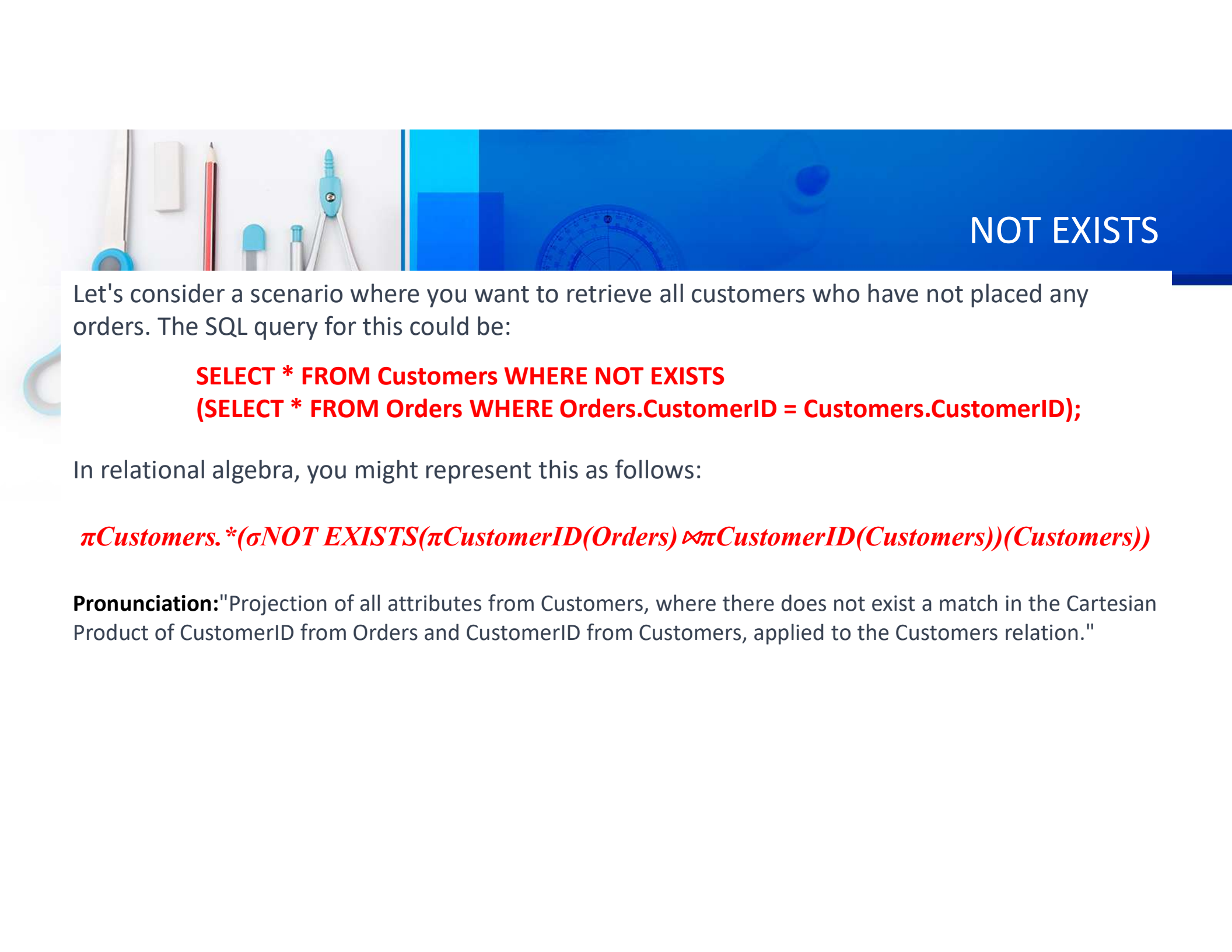
In relational algebra, you might represent this as follows:

```
 $\pi_{\text{Customers.}*}(\sigma_{\text{CustomerID} = \text{Orders.CustomerID}}(\text{Customers} \times \text{Orders}))$ 
```

**Pronunciation:** "Projection of all attributes from Customers, where there exists a tuple in the Cartesian Product of Customers and Orders such that the CustomerID matches between the two relations."

**Explanation:**

- $\times$  represents the Cartesian Product operation, combining all possible pairs of tuples from Customers and Orders.
- $\text{CustomerID} = \text{Orders.CustomerID}$  or  $\sigma_{\text{CustomerID} = \text{Orders.CustomerID}}$  represents the Selection operation, filtering the combined tuples where the CustomerID matches between Customers and Orders.
- $\text{Customers.}* \pi_{\text{Customers.}*}$  represents the Projection operation, selecting all attributes from the Customers relation for the final result.



## NOT EXISTS

Let's consider a scenario where you want to retrieve all customers who have not placed any orders. The SQL query for this could be:

```
SELECT * FROM Customers WHERE NOT EXISTS  
(SELECT * FROM Orders WHERE Orders.CustomerID = Customers.CustomerID);
```

In relational algebra, you might represent this as follows:

***$\pi_{Customers} . * (\sigma_{NOT\ EXISTS} (\pi_{CustomerID}(Orders) \bowtie \pi_{CustomerID}(Customers)) (Customers))$***

**Pronunciation:** "Projection of all attributes from Customers, where there does not exist a match in the Cartesian Product of CustomerID from Orders and CustomerID from Customers, applied to the Customers relation."



## NOT EXISTS

### Explanation:

Explanation:

- $\pi_{\text{CustomerID}}(\text{Orders})$  represents the Projection operation, selecting only the CustomerID attribute from the Orders relation.
- $\pi_{\text{CustomerID}}(\text{Customers})$  represents the Projection operation, selecting only the CustomerID attribute from the Customers relation.
- $\bowtie$  represents the Natural Join operation, combining all possible pairs of tuples from the two projections based on the common attribute CustomerID.
- $\text{NOT EXISTS}(\dots)$  represents the negation of the existence of tuples resulting from the join. If there is no match, the condition is satisfied.
- $\sigma_{\text{NOT EXISTS}(\dots)}$  represents the Selection operation with the NOT EXISTS condition.
- $\ast \pi_{\text{Customers}} \ast$  represents the Projection operation, selecting all attributes from the Customers relation for the final result.



## CREAT VIEW

Let's consider a scenario where you want to create a view named "HighValueOrders" that includes orders with a total amount greater than \$1000. The SQL query to create such a view might look like this:

```
CREATE VIEW HighValueOrders AS  
SELECT OrderID, CustomerID, OrderDate, TotalAmount FROM Orders  
WHERE TotalAmount > 1000;
```

In relational algebra, you might represent this as follows:

```
HighValueOrders ←  
πOrderID, CustomerID, OrderDate, TotalAmount(σTotalAmount > 1000(Orders))
```

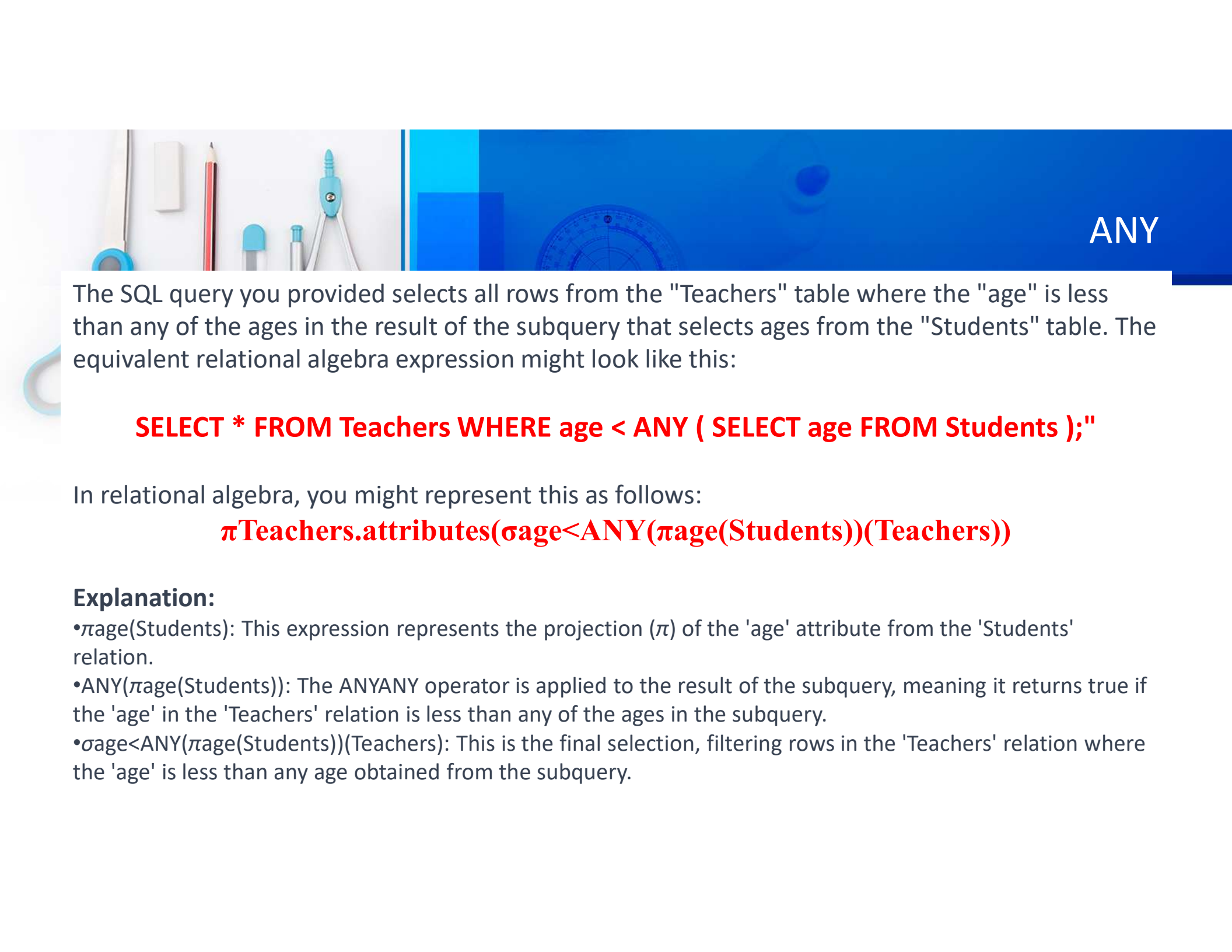
**Pronunciation:** Create a view named HighValueOrders, which is the projection of OrderID, CustomerID, OrderDate, and TotalAmount from the selection of Orders where TotalAmount is greater than 1000."

The header features a light blue background on the left with various stationery items: a pair of scissors, an eraser, a pencil, a sharpener, and a compass. On the right, there is a dark blue gradient background with faint geometric shapes, including a square and a circle.

## CREAT VIEW

### Explanation:

- $\sigma_{\text{TotalAmount} > 1000}(\text{Orders})$  represents the Selection operation, filtering the tuples from the Orders relation where the TotalAmount is greater than \$1000.
- $\pi_{\text{OrderID, CustomerID, OrderDate, TotalAmount}}$  represents the Projection operation, selecting specific attributes from the result of the selection.
- $\text{HighValueOrders} \leftarrow$  represents the assignment of the result to a view named HighValueOrders.



ANY

The SQL query you provided selects all rows from the "Teachers" table where the "age" is less than any of the ages in the result of the subquery that selects ages from the "Students" table. The equivalent relational algebra expression might look like this:

**SELECT \* FROM Teachers WHERE age < ANY ( SELECT age FROM Students );"**

In relational algebra, you might represent this as follows:

**$\pi_{\text{Teachers.attributes}}(\sigma_{\text{age} < \text{ANY}(\pi_{\text{age}}(\text{Students}))}(\text{Teachers}))$**

### Explanation:

- $\pi_{\text{age}}(\text{Students})$ : This expression represents the projection ( $\pi$ ) of the 'age' attribute from the 'Students' relation.
- $\text{ANY}(\pi_{\text{age}}(\text{Students}))$ : The ANY operator is applied to the result of the subquery, meaning it returns true if the 'age' in the 'Teachers' relation is less than any of the ages in the subquery.
- $\sigma_{\text{age} < \text{ANY}(\pi_{\text{age}}(\text{Students}))}(\text{Teachers})$ : This is the final selection, filtering rows in the 'Teachers' relation where the 'age' is less than any age obtained from the subquery.



The SQL query you provided selects all rows from the "Teachers" table where the "age" is less than all of the ages in the result of the subquery that selects ages from the "Students" table. The equivalent relational algebra expression might look like this:

**SELECT \* FROM Teachers WHERE age < ALL ( SELECT age FROM Students );"**

In relational algebra, you might represent this as follows:

**$\pi_{\text{Teachers.attributes}}(\sigma_{\text{age} < \text{ALL}(\pi_{\text{age}}(\text{Students}))}(\text{Teachers}))$**

### Explanation:

- $\pi_{\text{age}}(\text{Students})$ : This expression represents the projection ( $\pi$ ) of the 'age' attribute from the 'Students' relation.
- $\text{ALL}(\pi_{\text{age}}(\text{Students}))$ : The ALL operator is applied to the result of the subquery, meaning it returns true if the 'age' in the 'Teachers' relation is less than all of the ages in the subquery.
- $\sigma_{\text{age} < \text{ALL}(\pi_{\text{age}}(\text{Students}))}(\text{Teachers})$ : This is the final selection, filtering rows in the 'Teachers' relation where the 'age' is less than all ages obtained from the subquery.



Thank You all!